lecture 6 midterm this friday

review



selection: always takes $O(n^2)$
insertion: sorted $O(n)$ worst: $O(n^2)$
merge: always takes $O(n \log(n))$ but $O(n)$ space
quick: about $O(n \log(n))$ with $O(1)$ space
hash: $O(n \log(n))$ time

bottoms down at $O(n \log(n))$

Prove $\forall$ sorting algorithm A, WorstCaseTime(A) = $\Omega(n \log n)$
comparison lower bound

Previously, we did for this particular algorithm and analyze all inputs
how: look at all possible algorithms

Def: A comparison sorting algorithm sorts an array of numbers and it only way it can query the input is:
1. get the length
2. compare two elements
3. rearrange the elements

Not a comparison
- compare two elements
- everything in class

1. comparison sorting algorithm → sorting decision tree
2. sorting decision trees need to have a lot of nodes
3. they need to be deep
4. sorting algorithm can't be faster than $O(n \log(n))$

Only known way to get info

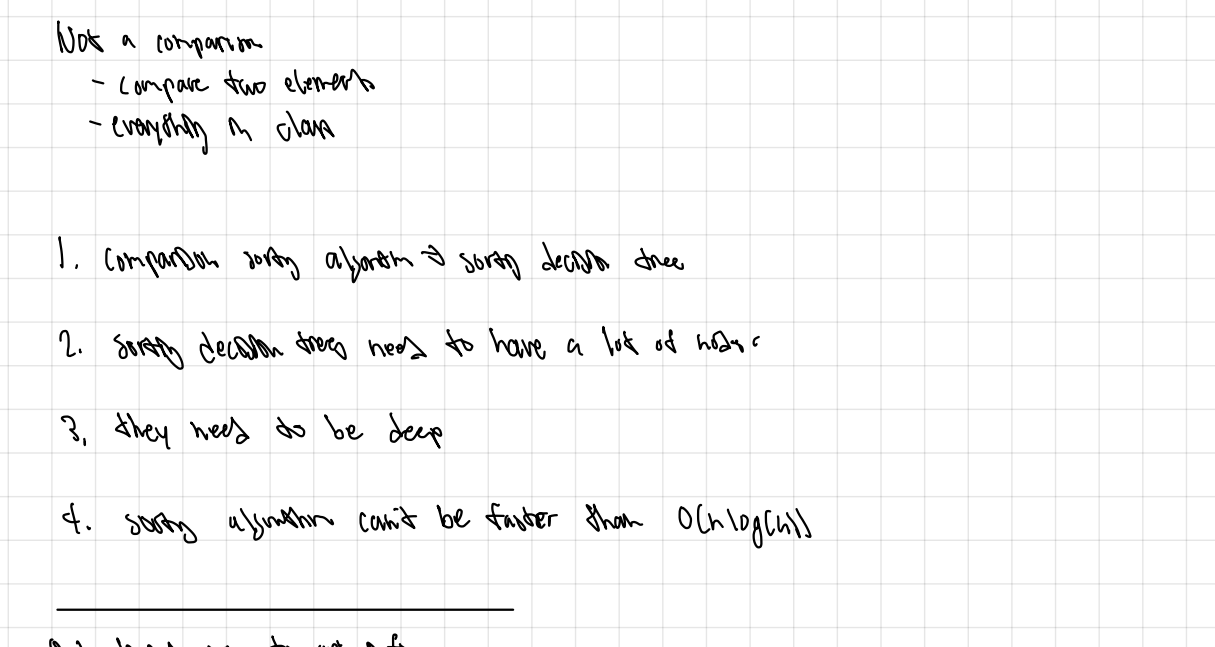
length 3 input: [a,b,c]

array can only get info after the first comparison

first comparison is always regardless of input. this is a characteristic of comparison algorithm

First index val < Zero index val

Decision Tree [A,B,C]



Def: A sorting decision tree is a binary tree where

1. Interior nodes are comparisons
2. Leaves are permutations that sort input based on comparisons or branching

This decision tree captures all of the decisions for selection sort

Lemma: Given any comparison sorting algorithm and a length n, you can make a sorting decision tree

---

Depth

leaves are h nodes from the root
each node is a comparison

every leave takes at least h nodes (comparisons) to get to the root

Convert algorithm to decision tree
leaves are permutations of the array

---

at least $n!$ leaves since each leaf is a permutation of the array
and any permutation might result in the sorted array

node and level estimation

with at least $n!$ nodes, since there are $n!$ leaves, a binary tree
has at least $\log(n!)$ levels

$\geq n!$ nodes



$\#\text{nodes} = 2^0 + 2^1 + 2^2 + \dots + 2^{l-1}$
$= 2^l - 1$

at least $n!$ nodes

nodes (lower bound) — leaves

$n! \geq 2^l - 1$
$n! + 1 \geq 2^l$
$\log_2(n!+1) \geq \log_2(2^l)$
$\log_2(n!+1) \geq l$

the best case is a balanced decision tree, $l = \log_2(\text{nodes})$

$\Omega(\log(n!)) \geq l$
lower bounds

$\log(n!) = \log(n(n-1)(n-2)\cdots(2)(1))$
$\geq \log(n(n-1)(n-2)\cdots(\frac{n}{2}))$
$\geq \log((\frac{n}{2})(\frac{n}{2})\cdots(\frac{n}{2}))$
$\geq \log((\frac{n}{2})^{\frac{n}{2}})$
$\geq \frac{n}{2}\log(\frac{n}{2})$

$\geq \Omega(n\log(n))$

$\Omega(n\log(n)) \geq l$

Given a comparison sorting algorithm, you can generate a decision tree.

The depth of the decision corresponds to a lower bound of the worst case runtime for the algorithm. The depth is at least $n\log(n)$ levels. Thus, the worst case needs at least $n\log(n)$ comparison/runtime

$n!$ leaves → at least $n!$ nodes → at least $\log_2(n!)$ levels → at least $n\log(n)$ levels → at least $n\log(n)$ comparisons

permutations — to the decision tree as binary tree